

Scenario

We are building an application for a **training institute**.

The training institute conducts multiple courses and needs a **Full-Stack Web Application** to manage its students and courses.

The requirements for the course management system are as follows:

- Save information about courses and students.
- Courses and students should have a **many-to-many relationship**.
- The application should have a rich UI which can perform:
 - Search
 - Filtering
 - Pagination
 - UI updates without reloading the application
- The application should be responsive and follow **Human Computer Interaction (HCI)** principles.
- The application should be able to use the user's device information such as:
 - Camera
 - Geolocation
 - Gyroscope
 - Other device APIs
- The application should expose a **REST API** that can be consumed by external services.
- The application should be optimized for performance.
- The application should have authentication.

The complete system will **not** be built in a single workshop.

Instead, we will progressively develop the application throughout the course.

The first step is to build the foundation of the application:

- Students
- Courses
- Forms
- Database operations
- CRUD functionality

Let's Start with Week 1

Week 1 focuses on building the first working version of our application.

At this stage, we are **not trying to build the final application**.

Instead, we want to focus on the simple question:

How can we build a web application that allows users to store and manage information?

To answer this question, we need to understand:

1. HTML Forms
2. HTTP Requests
3. Routes
4. Request Data
5. Validation
6. Database Interaction
7. CRUD Operations
8. HTML Responses
9. Blade Views

Problem Definition

The training institute needs to maintain information about:

- Students
- Courses

For Week 1, we will implement these as **two independent resources**.

The relationship between these entities will be added in later workshops.

Entity 1: Student

The following information about the student should be maintained.

Field	Description	Data Type
id	Unique identifier	Integer
name	Student's full name	String
email	Student's email address	String
phone	Student's contact number	String
address	Student's address	String
date_of_birth	Student's date of birth	Date

Field	Description	Data Type
created_at	Record creation time	DateTime
updated_at	Record modification time	DateTime

Student CRUD Operations

The application should provide the following operations.

Create Student

A form should allow the user to enter student information.

Read Students

The application should display a list of students in a table or list of cards.

The application should also provide a way to view an individual student's details.

Update Student

The user should be able to select an existing student and modify their information.

Delete Student

The user should be able to remove an existing student from the system.

Entity 2: Course

The following information about courses should be maintained.

Field	Description	Data Type
id	Unique identifier	Integer
name	Course name	String
description	Course description	String
duration	Course duration in weeks	Integer
fee	Course fee	Decimal
difficulty	Course difficulty	Enum

Field	Description	Data Type
is_active	Whether the course is currently provided	Boolean
created_at	Record creation time	DateTime
updated_at	Record modification time	DateTime

Difficulty Values

The `difficulty` field should support:

- Easy
- Medium
- Hard

Course CRUD Operations

The Course resource should support the same CRUD operations:

- Create Course
- List Courses
- View Course
- Edit Course
- Delete Course

Goal & Deliverables

By the end of Week 1, you should have a working Laravel application capable of:

- Managing Students
- Managing Courses
- Creating records
- Viewing records
- Editing records
- Deleting records
- Validating form input
- Storing information in MySQL

Week 1 Checklist

By the end of the workshop, you should be able to:

- Set up a Laravel development environment
 - Create a Laravel project
 - Configure a MySQL database
 - Create database tables using migrations
 - Create Laravel models
 - Define routes
 - Create Blade views
 - Create HTML forms
 - Handle HTTP requests
 - Receive and process form data
 - Validate user input
 - Store data in a database
 - Retrieve records
 - Update records
 - Delete records
 - Display database records using Laravel views
-

Development Environment Setup

Before starting the workshop, complete the development environment setup.

Follow: [Environment Setup Guideline](#)

The guide explains how to install and configure:

- VS Code
 - Git
 - WSL
 - PHP
 - Composer
 - Laravel
 - Node.js and NPM
 - MySQL
-

GitHub Repository Setup

Before starting development, create your GitHub repository.

Follow:

[GitHub Guideline](#)

For this workshop, use the repository and branch structure specified in the GitHub guideline.

Step 1: Create the Laravel Project

Open your WSL terminal.

Navigate to the directory where you created your workshop repository.

```
cd ~/6CS056/6CS056-Workshop
```

Switch to the Workshop1 branch:

```
git switch Workshop1
```

Create a new Laravel project in the folder:

```
laravel new ./
```

Open the project in VS Code:

```
code .
```

Start the Laravel Development Server

Run:

```
php artisan serve
```

You should see something similar to:

```
INFO  Server running on [http://127.0.0.1:8000].
```

Open the URL in your browser:

```
http://127.0.0.1:8000
```

You should see the Laravel welcome page.

Step 2: Configure the Database

Laravel stores database configuration in the project's `.env` file.

Open:

```
6CS056-Workshop/.env
```

Find the database configuration.

Example:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=workshopDB
DB_USERNAME=root
DB_PASSWORD=
```

Your values may be different.

For example:

- Your MySQL username may not be `root`.
- Your MySQL password may not be empty.
- Your MySQL server may use a different port.

Change these configuration values to reflect your actual database values

Create the Database

Create a MySQL database named:

```
workshopDB
```

Open the DBBeaver MySQL connection and run this SQL script to create the database:

```
CREATE DATABASE workshopDB;
```

Verify that the database exists:

```
SHOW DATABASES;
```

You should see:

```
WorkshopDB
```

Test the Laravel Database Connection

From the Laravel project directory, run:

```
php artisan migrate
```

If the database configuration is correct, Laravel should create its initial tables in DBBeaver.

You should see tables such as:

```
cache
cache_locks
jobs
job_batches
failed_jobs
migrations
password_reset_tokens
sessions
```

Step 3: Create the Student Model and Migration

We now need to create a database table for students.

Laravel can generate the model and migration together.

Run:

```
php artisan make:model Student -m
```

Laravel will create two important files.

Model

```
app/Models/Student.php
```

Migration

The migration will be inside:


```
database/migrations/
```

The filename will look similar to:

```
database/migrations/2026_09_21_123456_create_students_table.php
```

The exact timestamp in the filename will be different on your computer.

Step 4: Create the Students Table

Open the migration file:

```
database/migrations/xxxx_xx_xx_XXXXXX_create_students_table.php
```

Find the `up()` method.

Replace its contents with:

```
public function up(): void
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();

        $table->string('name');

        $table->string('email')->unique();

        $table->string('phone');

        $table->text('address')->nullable();

        $table->date('date_of_birth')->nullable();

        $table->timestamps();
    });
}
```

The complete migration should look similar to:

```
<?php
```

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('students', function (Blueprint $table) {
            $table->id();

            $table->string('name');

            $table->string('email')->unique();

            $table->string('phone');

            $table->text('address')->nullable();

            $table->date('date_of_birth')->nullable();

            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('students');
    }
};

```

Step 5: Run the Student Migration

Run:

```
php artisan migrate
```

Laravel should create the `students` table.

You can verify this using MySQL:

```
USE WorkshopDB;
```

```
SHOW TABLES;
```

You should see:

```
students
```

You can also inspect the table:

```
DESCRIBE students;
```

Step 6: Configure the Student Model

Open:

```
app/Models/Student.php
```

Update the model:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    protected $fillable = [
        'name',
        'email',
        'phone',
        'address',
        'date_of_birth',
    ];
}
```

Why do we need `$fillable` ?

We will later use:

```
Student::create($validated);
```

Laravel needs to know which fields can be assigned through mass assignment.

The `$fillable` property specifies the fields that are allowed.

Step 7: Create the Student Views

We now need to create the HTML pages for the Student resource.

Create the following directory:

```
resources/views/student/
```

Inside this directory create:

```
resources/views/student/list.blade.php
resources/views/student/create.blade.php
resources/views/student/detail.blade.php
resources/views/student/edit.blade.php
```

Step 8: Create Student Form

Open:

```
resources/views/student/create.blade.php
```

Add:

```
<!DOCTYPE html>
<html>
<head>
    <title>Create Student</title>
</head>
<body>

<h1>Create Student</h1>
```

```
<form action="/students" method="POST">

    @csrf

    <div>

        <label for="name">Name</label>

        <input
            type="text"
            id="name"
            name="name"
            value="{{ old('name') }}"
        >
    </div>

    <br>

    <div>

        <label for="email">Email</label>

        <input
            type="email"
            id="email"
            name="email"
            value="{{ old('email') }}"
        >
    </div>

    <br>

    <div>

        <label for="phone">Phone</label>

        <input
            type="text"
            id="phone"
            name="phone"
```

```
        value="{{ old('phone') }}"
    >
</div>

<br>

<div>
    <label for="address">Address</label>

    <textarea
        id="address"
        name="address"
    >{{ old('address') }}</textarea>
</div>

<br>

<div>
    <label for="date_of_birth">Date of Birth</label>

    <input
        type="date"
        id="date_of_birth"
        name="date_of_birth"
        value="{{ old('date_of_birth') }}"
    >
</div>

<br>

<button type="submit">
    Create Student
</button>

</form>

<br>
```

```
<a href="/students">Back to Students</a>
```

```
</body>
```

```
</html>
```

This to understand here:

- What is the role of the old function: For Example: `{{ old('name') }}`
- Why is POST used in the method

Step 9: Routes for create student

Open:

```
routes/web.php
```

Ensure following namespace are used:

```
use App\Models\Student;  
use Illuminate\Support\Facades\Route;  
use Illuminate\Http\Request;
```

Add the following Routes:

```
Route::get("/student/create", function () {  
    return view("student.create");  
});  
  
Route::post("/student", function (Request $request) {  
    $student_data = $request->input([  
        'name',  
        'email',  
        'phone',  
        'address',  
        'date_of_birth',  
    ]);  
    $student = Student::create($student_data);  
    return "Student {$student->name} created successfully!";  
});
```

```
});
```

Things to understand here:

- `Request $request` provides the access to the parsed HTTP Request
- `$request->input` provides access to the form inputs
- `Student::create($student_data)` run the SQL script to create the student

Step 10: Adding Validation in the Route

Open:

```
routes/web.php
```

Change the create student route by adding validation:

```
Route::post("/student", function (Request $request) {  
    $validated = $request->validate([  
        'name' => 'required|string|max:255',  
        'email' => 'required|email|max:255',  
        'phone' => 'required|string|max:20',  
        'address' => 'nullable|string|max:500',  
        'date_of_birth' => 'nullable|date',  
    ]);  
  
    $student = Student::create($validated);  
    return "Student {$student->name} created successfully!";  
});
```

Validation syntax

Laravel validation is written as an associative array, where the field name is the key and the validation rules are the value.

```
$request->validate([  
    'field_name' => 'rule1|rule2|rule3',  
]);
```

For example:


```
$request->validate([
    'name' => 'required|string|max:255',
]);
```

Here:

- `'name'` → the `name` attribute of the HTML form field.
- `=>` → connects the field with its validation rules.
- `'required|string|max:255'` → the rules that must be satisfied.
- `|` → separates multiple rules.
- `:` → provides a parameter to a rule, such as `max:255`.

Step 11 : Add Validation Messages Handling in the Create Form

Open:

```
resources/views/student/create.blade.php
```

In the form, display validation messages by adding:

```
@if($errors->any())
    <div>
        <h3>Please fix the following errors:</h3>

        <ul>
            @foreach($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif
```

Explanation

- This Blade code displays validation errors after Laravel redirects back to the form
- `@if($errors->any())`
Checks whether Laravel has **any validation errors**.
- `$errors`
Laravel provides this variable automatically to the Blade view. It contains the validation errors.

- `$errors->any()`
Returns `true` if there is at least one validation error.
- `@foreach($errors->all() as $error)`
Gets **all error messages** and loops through them one by one.
- `$error`
Contains the current error message.
- `{{ $error }}`
Displays the error message safely in HTML.
- `@endif`
Ends the `if` statement.

Step 12: Create the Student List Route

Open:

```
routes/web.php
```

Add:

```
Route::get('/students', function () {  
  
    $students = Student::all();  
  
    return view('student.list', [  
        'students' => $students  
    ]);  
});
```

Explanation:

- `/students` route displays the list of students
- To get all the students from database, use the function `Student::all()`
- The list of students from database is stored in `$students`
- You can pass `$students` to the view through the second argument of the `view()` function
- View accepts as **associative array** with a list of data

In the same file, edit the create student POST route to redirect to Student List view on success with success message:

```
Route::post("/student", function (Request $request) {
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email|max:255',
        'phone' => 'required|string|max:20',
        'address' => 'nullable|string|max:500',
        'date_of_birth' => 'nullable|date',
    ]);

    $student = Student::create($validated);

    return redirect()->route('students.index')
        ->with('success', "Student {$student->name} created successfully!");
});
```

Explanation:

- In Laravel, `with()` and `session()` work together using a concept called **Session Flashing**
- Session flashing means **storing data in the user's session for only one subsequent HTTP request** before it is automatically deleted.
- When you return a `redirect` and chain the `with` function, as `redirect()->with(key, value)` you can access the temporarily stored value using the `session(key)` function from the view
- Here, we wanted to indicate that user was successfully created with a message when we redirected to the Student List Page, we used the `redirect->with()` for that.

Step 12: Create the Student List View

Open:

```
resources/views/student/list.blade.php
```

Add:

```
<!DOCTYPE html>
<html>
<head>
    <title>Students</title>
</head>
<body>
```

```

<h1>Students</h1>

@if(session('success'))
    <p>{{ session('success') }}</p>
@endif

<a href="/students/create">Create Student</a>

<table border="1" cellpadding="10">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Phone</th>
            <th>Actions</th>
        </tr>
    </thead>

    <tbody>
        @forelse($students as $student)
            <tr>
                <td>{{ $student->id }}</td>
                <td>{{ $student->name }}</td>
                <td>{{ $student->email }}</td>
                <td>{{ $student->phone }}</td>

                <td>
                    <a href="/students/{{ $student->id }}">
                        View
                    </a>

                    <a href="/students/{{ $student->id }}/edit">
                        Edit
                    </a>
                </td>
            </tr>
        @endforelse
    </tbody>
</table>

```

```

        <form
            action="/students/{{ $student->id }}"
            method="POST"
            style="display:inline"
        >

            @csrf

            @method('DELETE')

            <button type="submit">
                Delete
            </button>
        </form>
    </td>
</tr>
@empty
    <tr>
        <td colspan="5">
            No students found.
        </td>
    </tr>
@endforelse
</tbody>
</table>

</body>
</html>

```

Things you need to understand here:

- `session('success')` : What is the use of this and where does the value come from?
- Where is the list of \$students coming from?
- Understand `@forelse` , `@empty` and `@endforelse`
- Understand the function of `{{ $student->name }}`
- What does `@method('DELETE')` and `@csrf` do

Step 10: Create the Student Detail Route & View

Open:

```
routes/web.php
```

Add a Route for Student Detail:

```
Route::get('/students/{id}', function ($id) {  
    $student = Student::findOrFail($id);  
  
    return view('student.detail', [  
        'student' => $student  
    ]);  
});
```

Explanation:

- `Route::get('/students/{id}', ...)` : Listens for GET requests on URLs like `/students/1` or `/students/99`. The `{id}` segment is captured as a route parameter.
- `function ($id)` : Passes the raw value of `{id}` from the URL into the function as a variable named `$id`.
- `$student = Student::findOrFail($id)` : Uses the Eloquent ORM to query the `students` table for a record with a matching primary key `$id`.
 - **If found:** It returns the `Student` model object.
 - **If not found:** It throws a `ModelNotFoundException`, which Laravel automatically converts into a **404 Not Found** response.
- `return view('student.detail', ...)` : Loads the Blade template located at `resources/views/student/detail.blade.php`.
- `['student' => $student]` : Passes the retrieved `$student` object to the view, making it accessible as `$student` in Blade.

Open:

```
resources/views/student/detail.blade.php
```

Add:

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
    <title>Student Details</title>  
</head>  
  
<body>
```

```
<h1>Student Details</h1>

<p>
  <strong>ID:</strong>
  {{ $student->id }}
</p>

<p>
  <strong>Name:</strong>
  {{ $student->name }}
</p>

<p>
  <strong>Email:</strong>
  {{ $student->email }}
</p>

<p>
  <strong>Phone:</strong>
  {{ $student->phone }}
</p>

<p>
  <strong>Address:</strong>
  {{ $student->address }}
</p>

<p>
  <strong>Date of Birth:</strong>
  {{ $student->date_of_birth }}
</p>

<a href="/students/{{ $student->id }}/edit">
  Edit Student
</a>
```

```
<br>

<a href="/students">
    Back to Students
</a>

</body>
</html>
```

Step 11: Create the Student Edit Form & Route

Open:

```
routes/web.php
```

Add a route for GET Edit form & to Handle POST Edit

```
Route::get('/students/{id}/edit', function ($id) {
    $student = Student::findOrFail($id);

    return view('student.edit', [
        'student' => $student
    ]);
});

Route::put('/students/{id}', function (Request $request, $id) {
    $student = Student::findOrFail($id);
    $validated = $request->validate([ 'name' => 'required|string|max:255',
        'email' => ['required', 'email', 'max:255',
Rule::unique('students')->ignore($student->id)],
        'phone' => 'required|string|max:20',
        'address' => 'nullable|string',
        'date_of_birth' => 'nullable|date', ]);

    $student->update($validated);
    return redirect('/students/' . $student->id)
        ->with('success', 'Student updated successfully!'); });
```


Explanation:

- `Rule::unique('students')->ignore($student->id)` : This rule is used for email validation
- Since email is a unique field, when a user changes their other details without changing their email, laravel will detect the email already exists and throw an exception.
- Adding the ignore rule tells Laravel to ignore the current student when checking for duplicate emails.

Open:

```
resources/views/student/edit.blade.php
```

Add:

```
<!DOCTYPE html>
<html>
<head>
    <title>Edit Student</title>
</head>
<body>

<h1>Edit Student</h1>

@if($errors->any())
    <div>
        <h3>Please fix the following errors:</h3>

        <ul>
            @foreach($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

<form
    action="/students/{{ $student->id }}"
    method="POST"
>
```

```
@csrf
@method('PUT')

<div>
    <label for="name">Name</label>

    <input
        type="text"
        id="name"
        name="name"
        value="{{ old('name', $student->name) }}"
    >
</div>

<br>

<div>
    <label for="email">Email</label>

    <input
        type="email"
        id="email"
        name="email"
        value="{{ old('email', $student->email) }}"
    >
</div>

<br>

<div>
    <label for="phone">Phone</label>

    <input
        type="text"
        id="phone"
        name="phone"
        value="{{ old('phone', $student->phone) }}"
    >
</div>
```

```
>
</div>

<br>

<div>
  <label for="address">Address</label>

  <textarea
    id="address"
    name="address"
  >{{ old('address', $student->address) }}</textarea>
</div>

<br>

<div>
  <label for="date_of_birth">Date of Birth</label>

  <input
    type="date"
    id="date_of_birth"
    name="date_of_birth"
    value="{{ old('date_of_birth', $student->date_of_birth) }}"
  >
</div>

<br>

<button type="submit">
  Update Student
</button>

</form>

<br>
```

```
<a href="/students">
    Back to Students
</a>

</body>
</html>
```

Step 12: Delete a Student

Add the route to web.php file:

```
Route::delete('/students/{id}', function ($id) {
    $student = Student::findOrFail($id);

    $student->delete();

    return redirect('/students')
        ->with(
            'success',
            'Student deleted successfully!'
        );
});
```

The route uses:

```
Route::delete()
```

because deleting a resource is represented by the HTTP `DELETE` method.

Step 13: Why Do We Use `@method('DELETE')` & `@method('PUT')` in the Edit & Delete Form?

HTML forms traditionally support:

```
GET
POST
```

They do not directly support:

PUT
PATCH
DELETE

Laravel provides method spoofing.

For example:

```
<form
  action="/students/5"
  method="POST"
>
  @csrf

  @method( 'DELETE' )

  <button type="submit">
    Delete
  </button>
</form>
```

The browser sends a POST request, but Laravel interprets it as a DELETE request.

Similarly, for updating:

```
<form
  action="/students/5"
  method="POST"
>
  @csrf

  @method( 'PUT' )

  ...
</form>
```

Laravel interprets this as a PUT request.

Step 14: Test Student CRUD

Start the server:

```
php artisan serve
```

Open:

```
http://127.0.0.1:8000/students
```

Test each operation.

Step 15: Similar to the Students Create CRUD for Courses

Step 16: Push the Code to GitHub

Before pushing your code, check the project status:

```
git status
```

Add the files:

```
git add .
```

Create a commit:

```
git commit -m "Complete Workshop 1 CRUD"
```

Push the branch:

```
git push origin workshop1
```

Use the branch name specified in the **GitHub Guideline** if your repository uses a different naming convention.

Verify your code on GitHub.

Step 43: Deploy the Workshop to the Student Server

Once the workshop is working locally, deploy the application to the student server.

Follow:

[Student Server Guideline](#)

The deployment process should include:

1. Push the project to GitHub.

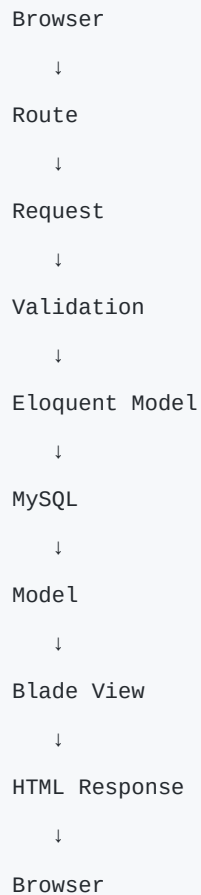
2. Clone the project on the student server.
3. Install PHP dependencies.
4. Configure the `.env` file.
5. Configure the database.
6. Run migrations.
7. Configure the application.
8. Test the deployed application.

Do not modify server-level configuration unless instructed by the server administrator.

What You Have Built

At the end of Week 1, you have built the foundation of the Training Institute Management System.

You have also learned the basic Laravel request flow:



Workshop 1 Completion Criteria

You have completed Workshop 1 when:

- Laravel is running locally.
 - MySQL is connected to Laravel.
 - `students` table exists.
 - `courses` table exists.
 - Student model works.
 - Course model works.
 - Student CRUD works.
 - Course CRUD works.
 - Student validation works.
 - Course validation works.
 - Student records are stored in MySQL.
 - Course records are stored in MySQL.
 - CSRF protection is used on forms.
 - Code has been committed to GitHub.
 - Application has been deployed according to the Student Server Guideline.
-

Key Concepts Learned

By completing this workshop, you should understand the relationship between:

HTML

Forms collect information from users.

HTTP

The browser sends requests to the server.

Laravel Routes

Routes determine what happens when a request reaches the application.

Request

Laravel provides access to submitted form data.

Validation

Laravel verifies that submitted data follows the application's rules.

Eloquent

Eloquent provides an object-oriented way of interacting with the database.

MySQL

MySQL permanently stores the application's data.

Blade

Blade generates the HTML response sent back to the browser.

CRUD

CRUD represents the four fundamental database operations:

Create

Read

Update

Delete

These concepts form the foundation for the more advanced Laravel features that will be introduced throughout the course.